CampusPrice: API Contract and Data Flow

Oct 29, 2026 · @BIT Project -PS team

Purpose

This contract fixes every request and response between the CampusPrice screens and the Python API, and says which data never leaves the server. It is a plan for the build team, not code; JSON is only the wire format.

The system has six partitions:

Partition	Role	Talks to
Owner console (React, laptop and tablet)	Menu Health, Demand Lab, Scenario Compare, Combo Studio, Recommendations	Python API, with the owner token
Student view (React, phone)	Student Budget Builder	Python API, with no key
Python API	Validates requests, applies the owner's guardrails, calls the models, shapes responses	Models, storage, LLM service
Model layers	Demand model, combo model, student budget layer, optimiser and Apply / Pilot / Hold gate	Storage (read), called by the API
Storage (SQLite or MySQL)	Items, sales log, student tiers, guardrails, override log	Models and API
LLM service (optional)	Turns a what-if into a scenario and explains computed numbers; never produces numbers	API only, with the server's own key

Two words in the question are read this way: "API key" means every credential and secret, and "float" means every numeric field and its type.

What lives where

Cost, margin, model internals and the LLM key never reach a student phone, and the owner sees ranges, never model internals.

Data	Lives on	Owner client sees	Student client sees
Item price, portion size, current combos	Server database	Yes	Yes (current menu only)
Unit cost, ₹ margin, margin %	Server database	Yes	No
Sales log (the 9 PS fields)	Server database	Aggregates only	No
Guardrails (margin floor, weekly change cap, student price cap, coverage target, combo discount cap, fairness mode)	Server database	Yes, editable	No
Student tiers (shares, budgets)	Server database	Aggregates	No
Model coefficients and posterior draws	Server only	Never; only ranges (mean, p10, p90)	No
Hidden truth from the generator	Grader only, not in the API	No	No
LLM provider key	Server environment only	Never	Never
Owner session token	Issued at login, held by the owner client	Yes	Not applicable
Student budget (one number)	Typed by the student, sent per request	Not applicable	Yes, with no identity attached

Credentials and keys

The system uses three credentials, and only one of them ever reaches a browser.

Credential	Held by	Where it is used	Reaches the browser
LLM provider key	Server environment	Server to LLM provider only	Never
Owner token	Owner client, issued at login	Authorization header on owner endpoints	Yes
Student access	No key	Student endpoints are read-only and rate-limited	Not applicable

Do not put secrets in front-end environment variables. Vite's `VITE_` and Create React App's `REACT_APP_` variables are bundled into the browser.

Conventions on every response

Every response wraps the payload as `data` plus `meta`. `meta` holds `model_version` (string), `data_source` (string, "synthetic" for the demo), `generated_at` (ISO timestamp) and `request_id` (string).

Kind	Type and rules
Money (price, cost, revenue, profit)	float, rupees, 2 decimals
Observed counts (qty_sold)	int
Predicted units	float; round only in the UI
portion_size_g, duration_days	int
discount_pct	float, 0 to 100 (as in the PS fields)
margin_pct	float, 0 to 100
Shares, probabilities, coverage, take rates	float, 0 to 1
Elasticity (price_effect)	float, negative

Kind	Type and rules
Range	object with floats `mean`, `p10`, `p90` (10th and 90th percentile across posterior draws)
IDs, day_type, status, flags	string (day_type: regular, exam, weekend, event)
fairness_mode, include_frontier, fallback	boolean
Dates	ISO 8601 strings

Endpoints by partition

The core plan has seven endpoints: items, predict, compare scenarios, evaluate combo, recommend, student offers and overrides. Three supporting ones are added (`/config`, `/assistant/query`, `/model/status`), and the frontier chart rides inside `/recommend`. Every endpoint except the student one needs the owner token.

Part	Method and path	Screen	Purpose
A	GET /items	Menu Health	Menu with margins, flags and portion tiers
A	GET /config, PUT /config	Menu Health	Read and edit the owner's guardrails
B	POST /predict	Demand Lab	Demand, revenue and profit for one changed input
C	POST /scenarios/compare	Scenario Compare	Today against named price and combo scenarios
D	POST /combos/evaluate	Combo Studio	One main plus add-on at a chosen discount
D	GET /combos/suggestions	Combo Studio	Best discount for each pairing
E	POST /recommend	Recommendations	Prices, portions and combos with an Apply, Pilot or Hold status

Part	Method and path	Screen	Purpose
E	POST /overrides, GET /overrides	Recommendations	Record and list the owner's decisions
F	GET /student/offers	Student Budget Builder	Profit-safe offers within a student's budget (no key)
G	POST /assistant/query	Any owner screen	Natural-language what-if and explanation
H	GET /model/status	Footer on every owner screen	Model version, training date, validation

A. Menu and settings (Menu Health)

GET /items takes no request fields. The response is:

Field	Type	Meaning
`items[].item_id`	string	Item identifier
`items[].name`	string	Display name
`items[].kind`	string	main or addon
`items[].price`	float	Current price, ₹
`items[].cost`	float	Unit cost, ₹
`items[].portion_size_g`	int	Current portion, grams
`items[].units_per_day`	float	Average units sold per day
`items[].margin_abs`	float	Margin per unit, ₹
`items[].margin_pct`	float	Margin, 0 to 100
`items[].flags[]`	string	loss_maker, poor_seller, star
`items[].portion_tiers[]`	object	`tier` (string), `portion_size_g` (int), `price` (float)
`combos[].combo_id`	string	Combo identifier
`combos[].combo_items[]`	string	Item ids in the combo

Field	Type	Meaning
`combos[].discount_pct`	float	Current discount, 0 to 100

GET /config and PUT /config are owner only. GET returns these guardrails and PUT sends them:

Field	Type	Meaning
`margin_floor_pct`	float	Lowest margin the optimiser may leave
`max_weekly_change_pct`	float	Largest price change allowed per week
`student_price_cap`	float	Student price cap, ₹ (raises the above_student_cap flag)
`coverage_target`	float	Share of students who can afford a complete meal, 0 to 1
`combo_discount_cap_pct`	float	Largest combo discount allowed
`fairness_mode`	boolean	No surge pricing on exam or event days

B. Demand Lab

POST /predict answers one question: what happens to demand and profit if the owner changes this input. The client sends:

Field	Type	Meaning
`item_id`	string	Item being changed
`price`	float	Proposed price, ₹
`portion_size_g`	int	Proposed portion, grams
`discount_pct`	float	Combo or offer discount, 0 to 100
`day_type`	string	regular, exam, weekend or event

The server returns:

Field	Type	Meaning
`units`	range	Predicted units per day
`revenue`	range	Predicted revenue, ₹ per day
`profit`	range	Predicted profit, ₹ per day
`margin_pct`	float	Margin at the proposed price
`price_effect`	range	Elasticity, always negative
`baseline`	object	`units`, `revenue`, `profit` (floats) for Today's menu
`curve[]`	object	`price`, `units_mean`, `profit_mean`, `profit_p10`, `profit_p90` (floats) across the allowed price band
`flags[]`	string	below_margin_floor, above_student_cap, change_too_large

C. Scenario Compare

POST /scenarios/compare puts named price and combo scenarios next to Today. The client sends:

Field	Type	Meaning
`scenarios[].name`	string	Scenario label
`scenarios[].item_changes[]`	object	`item_id` (string), `price` (float), `portion_size_g` (int)
`scenarios[].combo_changes[]`	object	`combo_id` (string), `discount_pct` (float)
`day_type_mix`	object, optional	Four floats (regular, exam, weekend, event) that sum to 1

The server returns `results[]`, with Today always first and the scenarios after it in the order sent. Each result has:

Field	Type	Meaning
`units`	range	Predicted units per day
`revenue`	range	Predicted revenue, ₹ per day
`profit`	range	Predicted profit, ₹ per day
`margin_pct`	float	Margin of the whole scenario
`coverage`	object	Students served within budget: `overall`, `tight`, `mid`, `flexible` (floats, 0 to 1)
`price_per_100g`	float	₹ per 100 g
`risk_prob_below_today`	float	Probability that profit lands below Today, 0 to 1
`delta_vs_today`	object	`units`, `revenue`, `profit` (floats)
`flags[]`	string	Guardrail warnings

D. Combo Studio

POST /combos/evaluate tests one main plus one add-on at a chosen discount. The client sends:

Field	Type	Meaning
`main_item_id`	string	Main item
`addon_item_id`	string	Add-on item
`discount_pct`	float	Combo discount, 0 to 100
`day_type`	string	regular, exam, weekend or event

The server returns:

Field	Type	Meaning
`take_rate`	range	Share of main buyers who take the combo, 0 to 1
`addon_lift`	float	Extra add-on sales caused by the combo

Field	Type	Meaning
`main_traffic_lift`	float	Extra main sales caused by the combo
`cannibalised_share`	float	Share of combo buyers who would have bought both anyway, 0 to 1
`break_even_take_rate`	float	Take rate at which the combo stops losing money, 0 to 1
`incremental_profit_per_day`	range	₹ per day against no combo
`prob_profit_improves`	float	Probability the combo adds profit, 0 to 1
`status`	string	apply, pilot or hold
`reason`	string	One-line explanation of the status

GET /combos/suggestions takes no request fields and returns `suggestions[]`:

Field	Type	Meaning
`main_item_id`	string	Main item
`addon_item_id`	string	Add-on item
`best_discount_pct`	float	Discount with the highest expected profit
`incremental_profit_per_day`	range	₹ per day at that discount
`status`	string	apply, pilot or hold

E. Recommendations and overrides

POST /recommend runs the optimiser inside the owner's guardrails. The client sends:

Field	Type	Meaning
`coverage_target`	float, optional	Overrides the saved target for this run, 0 to 1
`day_type_mix`	object, optional	Four floats that sum to 1
`include_frontier`	boolean	Return the profit-versus-affordability curve

The server returns:

Field	Type	Meaning
`recommendations[].rec_id`	string	Recommendation identifier
`recommendations[].kind`	string	price, portion or combo
`recommendations[].item_id` or `combo_id`	string	What the change applies to
`recommendations[].current` and `proposed`	float	Values in the unit of the kind (₹, grams or discount %)
`recommendations[].expected_profit_delta`	range	₹ per day
`recommendations[].prob_improves`	float	Probability profit improves, 0 to 1
`recommendations[].status`	string	apply (0.8 or above), pilot or hold
`recommendations[].reason`	string	One-line explanation
`recommendations[].guardrails_met[]`	string	Guardrails the proposal satisfies
`recommendations[].pilot_plan`	object, pilot only	`duration_days` (int, 7 by default) and `arms[]` of `price` (float) and `portion_size_g` (int)
`summary`	object	`profit_delta` (range), `coverage_before` and `coverage_after` (floats, 0 to 1)
`frontier[]`	object, when requested	`coverage` and `expected_profit` (floats)

POST /overrides (owner only) records the owner's decision on one recommendation. The client sends:

Field	Type	Meaning
`rec_id`	string	Recommendation being decided
`decision`	string	approve, edit or reject
`edited_value`	float, edit only	The owner's own value
`note`	string	Reason, kept for the next learning cycle

The server returns `override_id` (string), `status` (string) and `decided_at` (ISO timestamp).

GET /overrides (owner only) returns the log: the fields above plus `decided_by` (string).